

Theory of Computation (CSC262)

Chapter-wise Study & Priority Guide

BSc. CSIT – 4th Semester – Tribhuvan University

Based on TU Exam Papers: 2076, 2078, 2079, 2080, 2080(New), 2081, Model (2077 & 2082 papers not available at time of analysis)

■ ×1 Low – appeared once	■ ×2 Medium – appeared twice
■ ×3–4 High priority	■ ×5+ MUST PREPARE

★ = extremely high priority — prepare thoroughly, practice construction/derivation by hand.

Unit 1 – Basic Foundations (3 Hrs)

Alphabet & Closures (Kleene / Positive Closure, Power of Alphabet) ■ ×5 ★

Years: 2078, 2080, 2080(New), 2081, Model

Q: Define positive closure and Kleene closure with example; compute Σ^* , Σ^2 or closure of a given set (e.g. {ab}).

Basic String Definitions (alphabet, string, substring, prefix, suffix, concatenation, empty string/language) ■ ×2

Years: 2078, 2080

Q: Define alphabet, string, substring, prefix, suffix, concatenation of strings, empty string and empty language with examples.

■ **What to read:** Short, definition-heavy unit — just memorize the closure formulas cold (you will be asked to *compute* them, not just define them) and know the string/language vocabulary. 10 minutes of revision, not more.

Unit 2 – Introduction to Finite Automata (8 Hrs)

DFA / NFA Construction & Definitions (transition function, extended transition function, acceptance) ■ ×7 ★

Years: 2076, 2078, 2079, 2080, 2080(New), 2081, Model

Q: Construct a DFA or NFA over a given alphabet for a described language; show extended transition function or acceptance of a given input string.

NFA to DFA Conversion (Subset Construction) & Equivalence Proof ■ ×3

Years: 2076, 2079, 2080

Q: Show that for every NFA there is an equivalent DFA; convert a given NFA into DFA using subset construction.

ϵ -NFA / ϵ -Closure ■ ×4

Years: 2076, 2079, 2081, Model

Q: Define ϵ -closure of a state; convert a given regular expression or ϵ -NFA into an equivalent DFA/FA.

Moore Machine vs Mealy Machine ■ ×2

Years: 2080(New), 2081

Q: Differentiate between Moore and Mealy machines; design a Mealy machine for a given output condition.

■ **What to read:** This is the single highest-yield unit — DFA/NFA construction shows up in every paper. Practice constructing automata for language patterns (ends with X, contains substring X, even/odd count of symbols) until it is mechanical. Then layer on subset construction and ϵ -closure.

Unit 3 – Regular Expression (6 Hrs)

Regular Expression Construction (from a described language) ■ ×6 ★

Years: 2076, 2078, 2079, 2080, 2080(New), Model

Q: Write regular expression(s) over a given alphabet representing a described language (e.g. strings ending in a pattern, strings with specific symbol positions).

Pumping Lemma for Regular Languages (proving a language is NOT regular) ■ ×5 ★

Years: 2076, 2078, 2079, 2080, Model

Q: State and prove the Pumping Lemma; use it to show a given language (e.g. $L = \{0^n 1^n\}$) is not regular.

DFA Minimization — Table Filling Algorithm ■ ×2

Years: 2078, 2081

Q: Minimize a given DFA using the Table Filling Algorithm.

Regex ↔ FA Equivalence / Arden's Theorem ■ ×2

Years: 2076, 2080(New)

Q: Convert a regular expression to equivalent finite automata, or convert a DFA to its regular expression using Arden's Theorem.

■ **What to read:** Two must-drill skills here: (1) writing regex from a language description — practice translating conditions into patterns, and (2) the Pumping Lemma proof structure (assume regular $\rightarrow$ pick string of length $\geq p \rightarrow$ apply pumping $\rightarrow$ contradiction). Memorize the proof skeleton, not just the concept.

Unit 4 – Context Free Grammar (9 Hrs)

CFG to Chomsky Normal Form (CNF) Conversion ■ ×5 ★

Years: 2076, 2078, 2079, 2080, 2081

Q: Convert a given grammar to Chomsky Normal Form (removing useless symbols, null/unit productions where needed).

Parse Tree, Derivations & Ambiguity ■ ×4

Years: 2079, 2080, 2080(New), 2081

Q: Show leftmost/rightmost derivation and construct the parse tree for a given grammar and string; define ambiguous grammar and use a parse tree to demonstrate ambiguity.

Regular Grammar ↔ Finite Automata Equivalence ■ ×2

Years: 2079, 2081

Q: Convert a regular (right-linear/left-linear) grammar to an equivalent finite automaton.

Chomsky Hierarchy / Greibach Normal Form (GNF) ■ ×3

Years: 2076, 2078, 2079

Q: Explain the Chomsky Hierarchy of grammars and languages; define GNF and CNF with example.

Pumping Lemma for Context-Free Languages ■ ×1

Years: 2080(New)

Q: Use the pumping lemma for CFLs to prove a given language (e.g. $L = \{a^n b^n c^n\}$) is not context-free.

■ **What to read:** CNF conversion is a guaranteed numerical-style question — practice the full pipeline (remove ϵ , unit productions, useless symbols, then convert) on 3–4 different grammars until you don't need to think about the steps. Parse tree/derivation questions are easy marks if you're fast with leftmost/rightmost.

Unit 5 – Push Down Automata (7 Hrs)

PDA Construction (accepting a given language) ■ ×5 ★

Years: 2076, 2078, 2080, 2080(New), Model

Q: Construct a PDA accepting a described language over a given alphabet; show acceptance of a sample string via sequence of Instantaneous Descriptions (ID).

CFG to PDA Conversion ■ ×2

Years: 2079, 2081

Q: Given the formal definition of PDA, convert a context-free grammar into an equivalent PDA.

PDA to CFG Conversion ■ ×2

Years: 2080, 2080(New)

Q: Given a PDA (accepting by empty stack or final state), construct the equivalent context-free grammar.

PDA Acceptance: Final State vs Empty Stack (conversion between the two) ■ ×1

Years: 2076

Q: Explain how a PDA accepting by empty stack is converted into one accepting by final state, and vice versa.

■ **What to read:** PDA construction is the safest bet — practice designing PDAs for classic patterns (equal count of two symbols, palindromes, $a^n b^n$ style languages) and always show the ID trace, since that's usually asked explicitly. CFG↔PDA conversion is procedural — learn the standard construction rules, don't improvise.

Unit 6 – Turing Machine (10 Hrs)

TM Construction (accepting a given language) ■ ×6 ★

Years: 2076, 2078, 2079, 2080, 2080(New), Model

Q: Construct a Turing Machine accepting a described language; show transition diagram/table and test acceptance of a sample string via Instantaneous Description.

TM as Computing Function / Enumerator ■ ×3

Years: 2080, 2080(New), 2081

Q: Use a TM to compute a given function (e.g. $f(x) = 2x$, $f(n) = 0$), or as an enumerator of strings of a language.

Multi-tape / Multi-track TM & Storage in State ■ ×3

Years: 2076, 2080(New), Model

Q: Differentiate a multi-tape TM from a multi-track TM; explain restricted TMs (semi-infinite tape, multi-stack, storage in state).

TM Encoding & Universal Turing Machine (UTM) ■ ×2

Years: 2078, 2081

Q: Encode a given Turing Machine into binary form; explain the role of the Universal Turing Machine.

TM Definition, Roles & Variations ■ ×2

Years: 2078, 2079

Q: Define Turing Machine formally and explain its different variations and roles as a language recognizer.

■ **What to read:** Heaviest unit by hours (10) and it shows — TM construction alone appears in 6/7 papers. Build a mental checklist for TM design (states, tape symbols, blank symbol, transition table AND diagram) and always practice tracing the ID for the sample string given, since graders check the trace, not just the final machine.

Unit 7 – Undecidability and Intractability (5 Hrs)

Time & Space Complexity of TM (incl. Big-O, Big-Omega, Big-Theta) ■ ×4 ★

Years: 2076, 2078, 2079, 2080(New)

Q: Define time and space complexity of a Turing Machine; explain Big-O, Big-Omega and Big-Theta notation used for complexity measurement.

Tractable vs Intractable Problems ■ ×4

Years: 2076, 2078, 2079, 2080(New)

Q: Define tractable and intractable problems; discuss intractability with reference to a Turing Machine (e.g. is SAT intractable? Justify).

Undecidable Problems (Post's Correspondence Problem, Halting Problem) ■ ×2

Years: 2081, Model

Q: Define undecidable problem; discuss the Post Correspondence Problem, and argue why the Halting Problem is undecidable.

Complexity Classes (P, NP, NP-Complete) ■ ×2

Years: 2079, 2081

Q: Explain the complexity classes P, NP and NP-Complete and differentiate between them.

Abstract, Decision & Optimization Problems ■ ×1

Years: 2080

Q: Describe abstract, decision and optimization problems with examples.

■ **What to read:** Short unit, mostly conceptual — definitions and "explain why X is undecidable" style answers. Memorize the Halting Problem undecidability argument (diagonalization/contradiction) precisely, since it's asked almost verbatim across years. Complexity notations overlap with Unit 1 — revise them together.

Top Priority Summary (Sorted by Frequency)

Topic	Unit	Freq	Years
DFA / NFA Construction & Definitions	Finite Automata	×7	2076–81, Model
Regular Expression Construction	Regular Expression	×6	2076–80(N), Model
TM Construction (accepting a language)	Turing Machine	×6	2076–80(N), Model
Pumping Lemma (Regular Languages)	Regular Expression	×5	2076,78,79,80,Model
CFG to CNF Conversion	Context Free Grammar	×5	2076,78,79,80,81
PDA Construction	Pushdown Automata	×5	2076,78,80,80(N),Model
Alphabet & Closures (Kleene/Positive)	Basic Foundations	×5	2078,80,80(N),81,Model
Parse Tree, Derivations & Ambiguity	Context Free Grammar	×4	2079,80,80(N),81
ϵ -NFA / ϵ -Closure	Finite Automata	×4	2076,79,81,Model
TM Complexity (incl. Big-O/ Ω / Θ)	Undecidability	×4	2076,78,79,80(N)
Tractable vs Intractable Problems	Undecidability	×4	2076,78,79,80(N)
NFA to DFA Conversion	Finite Automata	×3	2076,79,80
Chomsky Hierarchy / GNF	Context Free Grammar	×3	2076,78,79
TM as Computing Function/Enumerator	Turing Machine	×3	2080,80(N),81
Multi-tape/track TM & Storage in State	Turing Machine	×3	2076,80(N),Model

Final Exam Very High Probability Topics (Never Miss These)

MUST PREPARE (×5+, near-certain): DFA/NFA construction, Regular expression construction, TM construction, Pumping Lemma (regular), CFG to CNF, PDA construction, Alphabet & closures.

High priority (×3–4): Parse tree/derivations/ambiguity, ϵ -NFA/ ϵ -closure, TM complexity & Big-O notation, Tractable/Intractable problems, Chomsky Hierarchy.

Numerical / construction-heavy (practice by hand, not just theory): DFA/NFA/PDA/TM design for a given language, CNF conversion steps, subset construction, table-filling minimization.

Due for a repeat (asked before, quiet the last cycle — good bet): Arden's Theorem, PDA acceptance (final state vs empty stack conversion), Abstract/Decision/Optimization problems, Moore vs Mealy machine.